

Accelerated Linearized Laplace Approximation for Bayesian Deep Learning

Stepanov Ilya

Moscow Institute of Physics and Technology

2025

Bayesian Neural Networks: Basic Concepts

Problem with Neural Networks

- ▶ Neural networks $g_{\theta}(x)$ find only the **most probable** interpretation of data
- ▶ **Do not account for uncertainty**: prone to overfitting and overconfidence

Solution: Bayesian Approach

- ▶ Introduce **prior distribution** $p(\theta)$ on parameters
- ▶ Seek **posterior distribution**:

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}$$

- ▶ Analytically intractable due to neural network nonlinearity

Inference and Prediction in BNN

Approximate Inference Methods

Find approximation $q(\theta) \approx p(\theta|D)$:

- ▶ Variational Inference (VI)
- ▶ Laplace Approximation (LA)
- ▶ MCMC methods
- ▶ POVI

BNN Prediction

For new x^* prediction:

$$p(y|x^*, D) = \mathbb{E}_{p(\theta|D)} p(y|x^*, \theta) \approx \mathbb{E}_{q(\theta)} p(y|x^*, \theta) \approx \frac{1}{S} \sum_{s=1}^S p(y|g_{\theta_s}(x^*))$$

where $\theta_s \sim q(\theta)$ - parameter samples

Laplace Approximation (LA)

Laplace Approximation (LA)

Builds Gaussian approximate posterior:

$$q(\theta) = \mathcal{N}(\theta; \hat{\theta}, \Sigma)$$

where:

- ▶ $\hat{\theta} = \arg \max_{\theta} (\log p(D|\theta) + \log p(\theta))$ (MAP estimate)
- ▶ $\Sigma^{-1} = -\nabla_{\theta\theta}^2 (\log p(D|\theta) + \log p(\theta))|_{\theta=\hat{\theta}}$ (Hessian)

With Gaussian Prior

For $p(\theta) = \mathcal{N}(\theta; 0, \sigma_0^2 I_P)$:

$$-\nabla_{\theta\theta}^2 \log p(\theta)|_{\theta=\hat{\theta}} = I_P / \sigma_0^2$$

Generalized Gauss-Newton (GGN) Approximation

GGN Approximation

Due to Hessian intractability:

$$\Sigma^{-1} = \sum_{i=1}^N J_{\hat{\theta}}(x_i)^{\top} \Lambda(x_i, y_i) J_{\hat{\theta}}(x_i) + I_P / \sigma_0^2$$

where:

- ▶ $J_{\hat{\theta}}(x) = \nabla_{\theta} g_{\theta}(x)|_{\theta=\hat{\theta}}$ (Jacobian)
- ▶ $\Lambda(x, y) = -\nabla_{gg}^2 \log p(y|g)|_{g=g_{\hat{\theta}}(x)}$ (Likelihood Hessian)

Scalability Challenges

The GGN (Generalized Gauss–Newton) matrix of size $P \times P$ is intractable for large networks.

- ▶ **Diagonal approximation:** keeps only diagonal elements
- ▶ **KFAC approximation:** block-diagonal structure via Kronecker factorization
- ▶ **Subspace methods:** reduce dimensionality (e.g., last layer only)

Linearized Laplace Approximation (LLA)

Idea

To align prediction, use a linearized model:

$$g_{\theta}^{\text{lin}}(x) = g_{\hat{\theta}}(x) + J_{\hat{\theta}}(x)(\theta - \hat{\theta})$$

Approximating Distributions

- ▶ **Parameter posterior:** $p(\theta|D) \approx \mathcal{N}(\theta; \hat{\theta}, \Sigma)$
- ▶ **Likelihood:** $p(y^*|\theta, x^*) \approx \mathcal{N}(y^*; g_{\hat{\theta}}(x^*) + J_{\hat{\theta}}(x^*)(\theta - \hat{\theta}), \sigma_1^2)$

Bayesian Predictive Distribution

Using linear-Gaussian model properties:

$$p(y^*|x^*, D) \approx \mathcal{N}\left(y^*; g_{\hat{\theta}}(x^*), \sigma_1^2 + J_{\hat{\theta}}(x^*)\Sigma J_{\hat{\theta}}(x^*)^T\right)$$

Gaussian Process View of LLA

From Parameter Space to Function Space

We move from a parameter-space view to a **function-space Gaussian process (GP)**:

$$q(f) = \text{GP}(f \mid g_{\hat{\theta}}(x), \kappa_{\text{LLA}}(x, x'))$$

with kernel

$$\kappa_{\text{LLA}}(x, x') = J_{\hat{\theta}}(x) \Sigma J_{\hat{\theta}}(x')^{\top}$$

- LLA represents a neural network as a Gaussian Process around its MAP estimate.

Neural Tangent Kernel

Connection to Neural Tangent Kernel (NTK)

Using the Woodbury identity, the covariance matrix can be rewritten as:

$$\Sigma = \sigma_0 I_P - J_{\hat{\theta}, X} \left[\Lambda_{X, Y} / \sigma_0 + J_{\hat{\theta}, X} J_{\hat{\theta}, X}^\top \right]^{-1} J_{\hat{\theta}, X}^\top$$

Substituting into κ_{LLA} yields:

$$\begin{aligned} \kappa_{\text{LLA}}(x, x') = \sigma_0^2 \Big[& \kappa_{\text{NTK}}(x, x') - \kappa_{\text{NTK}}(x, X) \left(\Lambda_{X, Y}^{-1} / \sigma_0^2 + \kappa_{\text{NTK}}(X, X) \right)^{-1} \\ & \cdot \kappa_{\text{NTK}}(X, x') \Big] \end{aligned}$$

where

$$\kappa_{\text{NTK}}(x, x') = J_{\hat{\theta}}(x) J_{\hat{\theta}}(x')^\top$$

Scalability Limitations of LLA

Main Bottleneck

Computing and inverting the matrix

$$\kappa_{\text{NTK}}(X, X) \in \mathbb{R}^{(N \cdot C) \times (N \cdot C)}$$

is computationally infeasible when N or C is large.

→ **Solution: Accelerated kernel Approximation for LLA (ELLA).**

Nyström Approximation for NTK

Main Idea

We approximate the NTK using a low-rank feature map:

$$\kappa_{\text{NTK}}(x, x') \approx \phi(x)\phi(x')^\top,$$

where $\phi : \mathcal{X} \rightarrow \mathbb{R}^{C \times K}$ captures the top- K spectral components of the NTK.

- ▶ Select M landmark pairs $\tilde{X} = \{(x_m, i_m)\}_{m=1}^M$.
- ▶ Compute matrix

$$K = J_{\hat{\theta}, \tilde{X}} J_{\hat{\theta}, \tilde{X}}^\top$$

from Jacobians of selected points.

- ▶ Obtain top- K eigenpairs (λ_k, u_k) of K .
- ▶ Define feature map:

$$\phi(x) = [J_{\hat{\theta}}(x)v_1, \dots, J_{\hat{\theta}}(x)v_K], \quad v_k = \frac{J_{\hat{\theta}, \tilde{X}}^\top u_k}{\sqrt{\lambda_k}}.$$

Accelerating LLA via Kernel Approximation (ELLA)

Compact kernel form

Using $\phi(x)$, we approximate the predictive covariance of LLA:

$$\kappa_{\text{ELLA}}(x, x') = \phi(x) \left[\sum_i \phi(x_i)^\top \Lambda(x_i, y_i) \phi(x_i) + \frac{I_K}{\sigma_0^2} \right]^{-1} \phi(x')^\top$$

- ▶ Reduces $P \times P$ inversion to $K \times K$ ($K \ll P$).
- ▶ Preserves the GP interpretation of LLA.
- ▶ Enables fast uncertainty estimation with pretrained networks.
- ▶ Empirically stable for $K < 100$.

ELLA Implementation Overview

Algorithm 1: Estimate Posterior $q(f)$

- ▶ Compute $G = \sum_i \phi(x_i)^\top \Lambda_{x_i, y_i} \phi(x_i) + I_K / \sigma_0^2$.
- ▶ Invert: G^{-1} .
- ▶ Posterior predictive $q(f(x)) = \mathcal{N}(g_{\hat{\theta}}(x), \phi(x) G^{-1} \phi(x)^\top)$

Algorithm 2: Build Feature Map ϕ (Nyström Approximation)

- ▶ Select M random landmark pairs (x_m, i_m) .
- ▶ Compute Jacobians $J_{\hat{\theta}, \tilde{X}}[m] = \nabla_{\hat{\theta}} g_{\hat{\theta}}(x_m)[i_m]$.
- ▶ Perform eigendecomposition of $K = J_{\hat{\theta}, \tilde{X}} J_{\hat{\theta}, \tilde{X}}^\top$.
- ▶ Define $v_k = J_{\hat{\theta}, \tilde{X}}^\top u_k / \sqrt{\lambda_k}$.
- ▶ Build $\phi(x) = [J_{\hat{\theta}}(x)v_1, \dots, J_{\hat{\theta}}(x)v_K]$.

Experimental Results: Uncertainty in ELLA

Test accuracy and ECE on CIFAR-10

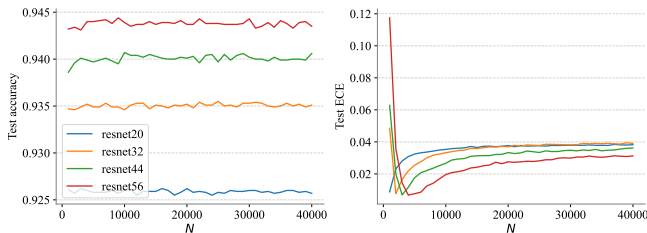


Рис.: Test accuracy and ECE of ELLA vs. number of training samples N

* Expected Calibration Error (ECE):

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|$$

M : bins, n : samples, $|B_m|$: bin size, $\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{I}(\hat{y}_i = y_i)$: bin accuracy, $\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$: bin confidence

Experimental Results: Uncertainty in ELLA

Test NLL on CIFAR-10

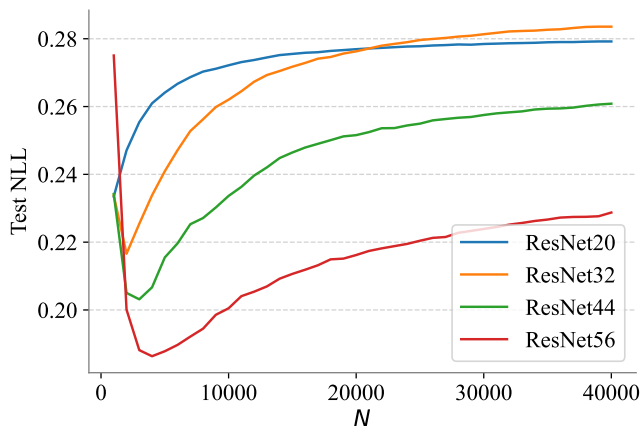


Рис.: Test NLL of ELLA vs. number of training samples N .

Experimental Results: Regression

Regression problem

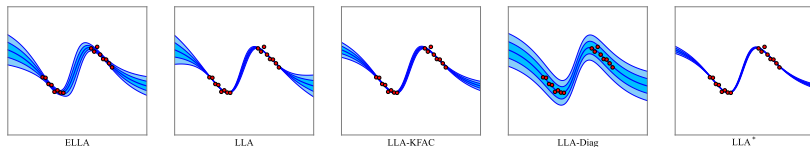


Рис.: Regression on $y = \sin(2x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.2)$. Red dots, central blue curves, and shaded regions refer to the training data, mean predictions, and uncertainty respectively. The model is a pretrained multilayer perceptron (MLP) with 3 hidden layers.

Таблица: Comparison on the approximation error to vanilla LLA in the illustrative regression case, measured by the KL divergence between the predictive distributions.

Method	ELLA	LLA-KFAC	LLA-Diag	LLA _*
KL div.	0.83	0.35	1.71	2.44

Experimental Results: Classification

Таблица: Comparison on CIFAR-10 test accuracy (%) $\uparrow$, NLL $\downarrow$, and ECE $\downarrow$. Average over 5 runs.

Method	ResNet-20			ResNet-32			ResNet-44			ResNet-56		
	Acc	NLL	ECE	Acc	NLL	ECE	Acc	NLL	ECE	Acc	NLL	ECE
<i>ELLA</i>	92.5	0.233	0.009	93.5	0.215	0.008	93.9	0.204	0.007	94.4	0.187	0.007
<i>MAP</i>	92.6	0.282	0.039	93.5	0.292	0.041	94.0	0.275	0.039	94.4	0.252	0.037
<i>MFVI-BF</i>	92.7	0.231	0.016	93.5	0.222	0.020	93.9	0.206	0.018	94.4	0.188	0.016
<i>LLA*</i>	92.6	0.269	0.034	93.5	0.259	0.033	94.0	0.237	0.028	94.4	0.213	0.022
<i>LLA*-KFAC</i>	92.6	0.271	0.035	93.5	0.260	0.033	94.0	0.232	0.028	94.4	0.202	0.024
<i>LLA-Diag</i>	92.2	0.728	0.404	92.7	0.755	0.430	92.8	0.778	0.445	92.9	0.843	0.480
<i>LLA-KFAC</i>	92.0	0.852	0.467	91.8	1.027	0.547	91.4	1.091	0.566	89.8	1.174	0.579

Experimental Results: Classification

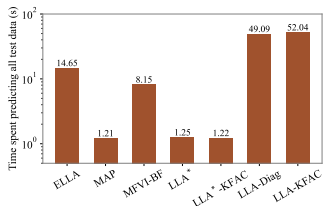


Рис.: Comparison on the wall-clock time used for predicting all CIFAR-10 test data (measured on an NVIDIA A40 GPU) with ResNet-20.

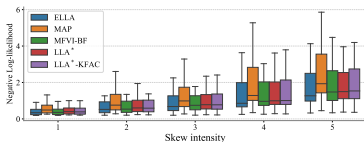


Рис.: NLL on CIFAR-10 corruptions for models trained with ResNet-56 architecture.

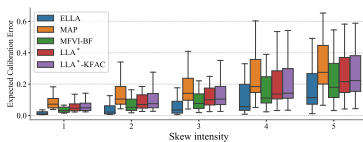


Рис.: ECE on CIFAR-10 corruptions for models trained with ResNet-56 architecture.

Experimental Results: Classification

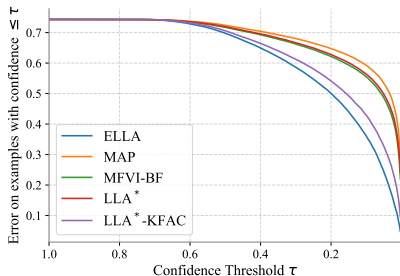


Рис.: Error versus confidence plots for methods trained on CIFAR-10 with ResNet-20 and tested on CIFAR-10+SVHN.

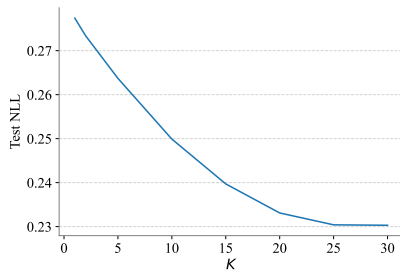


Рис.: Test NLL of ELLA varies w.r.t. K on CIFAR-10 with ResNet-20.

Experimental Results: Classification

Method	ResNet-18			ResNet-34			ResNet-50		
	Acc.	NLL	ECE	Acc.	NLL	ECE	Acc.	NLL	ECE
<i>ELLA</i>	69.8	1.243	0.015	73.3	1.072	0.018	76.2	0.948	0.018
<i>MAP</i>	69.8	1.247	0.026	73.3	1.081	0.035	76.2	0.962	0.037
<i>MFVI-BF</i>	70.3	1.218	0.042	73.7	1.043	0.033	76.1	0.945	0.030

Таблица: Comparison on test accuracy (%) $\uparrow$, NLL $\downarrow$, and ECE $\downarrow$ on ImageNet. Average over 3 runs.

Method	Acc.	NLL	ECE
<i>ELLA</i>	81.6	0.695	0.022
<i>MAP</i>	81.5	0.700	0.039

Таблица: Comparison on test accuracy (%) $\uparrow$, NLL $\downarrow$, and ECE $\downarrow$ on ImageNet with ViT-B architecture.

Key Ideas

Problem

- ▶ Existing methods: computationally expensive or poor quality

ELLA Solution

- ▶ Kernel approximation (Nyström) for linearized Laplace
- ▶ Maintains quality with computational efficiency

Validation

- ▶ Strong theoretical guarantees provided
- ▶ Competitive accuracy with better calibration (lower ECE)